

Introduction

Designing a secure protocol in which point A is able to communicate with point B is crucial especially within a banking system. The goal of this project is to develop a key distribution protocol along with key derivation functions in order to generate and use unique keys to prevent data modification and attacks on the communication channel.

A common attack on communication channels like the one in this project is replay attacks. Replay attacks are where attackers will intercept a message and find ways to either modify or attach their own message to send to the receiver in hopes of tricking the receiver into thinking that they are the sender. Combating these sorts of attacks involves implementing methods to allow the receiver to verify the integrity of the message as well as the recency of the message.

This project is able to demonstrate how an authenticated key distribution protocol is built as well as how different keys are derived and generated in real time. This project also includes a very basic GUI as well as a very simple banking system that includes deposit, withdrawal, and check balance functions.

Design

This project is built in Java and implements techniques and principles such as nonces, SecretKeys, Hashed message authentication code (HMAC) functions and HMAC based key distribution functions (KDF).

Below is a rundown of each implemented function as well as how it is applied to the working of the project. All the functions listed are shared between the ATM and the BankServer thread code.

```
private static void generateNonce () {  
    SecureRandom n = new SecureRandom();  
    byte[] nBytes = new byte[5];  
    n.nextBytes(nBytes);  
    nonce = Base64.getEncoder().encodeToString(nBytes);  
}
```

This function generates a nonce. Nonce is a number used once and is used to prevent replay attacks since it is only used in the current session.

```
private static SecretKey generateSecret(String key) {
    try {
        SecretKeyFactory fact = SecretKeyFactory.getInstance("DES");
        byte[] b = key.getBytes();
        return fact.generateSecret(new DESKeySpec(b));
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}
```

This function is used to generate a SecretKey that is later used in the encoding and decoding functions. The function requires a string which will be used to generate the SecretKey with.

```
public static void encodeDES (String message, SecretKey s) {
    try {
        ciph = Cipher.getInstance("DES/ECB/PKCS5Padding");
        ciph.init(Cipher.ENCRYPT_MODE, s);
        encrypted = ciph.doFinal(message.getBytes());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

This function is used to encode the given message with the given SecretKey using Data Encryption Standard (DES) algorithm.

```
public static void encodeDES (byte[] message, SecretKey s) {
    try {
        ciph = Cipher.getInstance("DES/ECB/PKCS5Padding");
        ciph.init(Cipher.ENCRYPT_MODE, s);
        encrypted = ciph.doFinal(message);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Same function as above except it takes a message as a byte array instead of a string.

```
public static void decodeDES (byte[] message, SecretKey s) {
    try {
        ciph = Cipher.getInstance("DES/ECB/PKCS5Padding");
        ciph.init(Cipher.DECRYPT_MODE, s);
        decrypted = ciph.doFinal(message);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

This function decodes the given message with the given SecretKey using the DES algorithm.

```

private static void masterGenerate (String message) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] b = digest.digest(message.getBytes());
        StringBuilder hexString = new StringBuilder(2 * b.length);
        for (int i = 0; i < b.length; i++) {
            String hex = Integer.toHexString(0xff & b[i]);
            if(hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        masterString = hexString.toString();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

This function generates a masterString that is used to derive the encryption key and the mac key later on. This function takes a string and puts it through the SHA-256 algorithm and produces a string of hexadecimals. This string of hexadecimals is later used to generate a masterKey.

```

private static void addToLog (String message) {
    try {
        LocalDateTime ldt = Instant.ofEpochMilli(ts.getTime()).atZone(ZoneId.systemDefault()).toLocalDateTime();
        String m = message + "|" + ldt;
        encodeDES(m, sec);
        String s = Base64.getEncoder().encodeToString(encrypted);
        FileWriter file = new FileWriter("latest.log", true);
        BufferedWriter write = new BufferedWriter(file);
        write.write(s);
        write.newLine();
        write.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

This function is used to add the given message to the given audit logs. It takes the message and attaches a LocalDateTime displayed in YYYY-MM-DDTHH:MM:SS. This message is then encoded to protect the information and it is then appended to the latest.log file.

```

public static String hmac(String data, String key){
    try {
        SecretKeySpec secretKeySpec = new SecretKeySpec(key.getBytes(), "HmacSHA256");
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(secretKeySpec);
        byte[] b = mac.doFinal(data.getBytes());
        String r = "";
        for (byte i : b) {
            r += String.format("%02X", i);
        }
        return r;
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}

```

This function takes the given data and key and runs it through the HMAC function to produce a MAC string that is used in validating the data that is sent from point A to point B.

```

private static boolean verifyMac(String[] arr) {
    if (arr[arr.length - 1].compareTo(hmac("macKey", masterString)) == 0) {
        if (new Timestamp(System.currentTimeMillis()).getTime() - Long.parseLong(arr[arr.length - 2]) < 1500)
            return true;
    }
    return false;
}

```

This function takes the given message and verifies the timestamp as well as the MAC string to ensure data validity.

Key Authentication Protocol

The protocol that was developed in this project involved the use of nonces and timestamps to verify and validate the message being sent back and forth during the key authentication protocol. The following steps explain how the messages being exchanged are valid.

1. ATM generates a nonce and attaches a timestamp which is then encrypted with the sharedKey that is already known by both server and client. This message has been protected using 3 different methods. Using a nonce is the first one and it provides uniqueness to this message compared to other messages. The added timestamp also provides security as the receiver can compare the timestamp from when it was sent to when it was received. Finally, the message is encrypted with a key that is shared between the server and the client.
2. Server receives the message and decrypts it using the sharedKey. After decrypting, it is able to validate the timestamp included in this message and verify the message integrity. The server then generates a nonce and attaches its nonce with the ATM nonce along with a timestamp which is then all encrypted with the sharedKey. The server also generates a tempKey using its nonce and encrypts that with the sharedKey.

3. ATM then receives the message including both nonces and the server timestamp encrypted along with the encrypted tempKey. ATM decrypts all of it, verifies its nonce and timestamp then extracts the temp key for use. ATM then encrypts the server's nonce with the tempKey and sends it back to the server to validate its connection and verify that it was able to extract the tempKey.
4. Server receives the final message back and decrypts it using tempKey. This allows the server to confirm that it is indeed talking to the ATM and that the connection is secure using the keys that were generated.
5. Creating the master key involved appending the 2 nonces together and putting it through the SHA256 algorithm to produce a key that is both encrypted and session unique. This key is used to derive an encryption key and a MAC string used for data validation.
6. To verify that both ends have generated the same master key, they send each other the master key encrypted with the master key through the DES algorithm. If either side can decrypt the message and verify the contents, then they have generated the same master keys and can continue with regular operations.

Key derivation method

The key derivation method used in this project involves the use of HMAC function and a set string to derive the encryption key and the MAC key. The encryption key is generated by running the master string and the word "encryption" as the key and data field respectively. This method is applied the same with the MAC key and instead of "encryption", it is "macKey". The resulting MAC string is used to append onto messages and to verify that the data in the message is valid and has not been tampered with. The resulting encryption string is used to generate another secretKey which will be used to encrypt the data that is communicated between the server and ATM from this step on.

General program functions

The program works by creating threads to handle each individual ATM connecting to the server.

```

public class BankServer {
    static ArrayList<BankThread> threads = new ArrayList<BankThread>();
    static ArrayList<clientFile> clients = new ArrayList<clientFile>();

    public static ArrayList<clientFile> getClients () {
        return clients;
    }
    public static void main(String[] args) {
        int port = 50000;
        ServerSocket s;
        Socket server;
        clients.add(new clientFile("Andy|password"));
        clients.add(new clientFile("Tester|password"));
        clients.add(new clientFile("ClientC|password"));
        try {
            s = new ServerSocket(port);
            while (true) {
                server = s.accept();
                BankThread thread = new BankThread(server);
                threads.add(thread);
                thread.start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

The server has an arrayList of threads that handle each individual connection as well as an arrayList of clients. Test clients have been created within the program to simulate operations for demonstration purposes.

```

String message = new String(decrypted);
String[] arr = message.split("\\|");
String nonceA = arr[0];
if (Long.parseLong(arr[1]) - new Timestamp(System.currentTimeMillis()).getTime() > 20) {
    System.out.println("Timestamp invalid, closing connection");
    sock.close();
    return;
}

```

Both the ATM and the server have these lines of code and its main and most important function is to verify the validity of the message based on the timestamps. The message is received and each part of the message is separated by “|”. This makes it easier to just split the message and use parts of a message by iterating through an array. It compares the timestamps and if the timestamp of the received message is outside of the threshold, the message is deemed invalid and is rejected. The connection channel is then closed and no further communications will be accepted.

```

masterGenerate(nonceA + nonce);
masterSec = generateSecret(masterString);
encryptedIn = (byte[]) in.readObject();
decodeDES(encryptedIn, masterSec);
if (masterString.compareTo(new String(decrypted)) != 0){
    System.out.println("Master Secret Invalid, closing connection");
    sock.close();
    return;
}
encodeDES(masterString, masterSec);
out.writeObject(encrypted);

```

These lines of code show the generation of the master key. Both the server and the ATM nonce are appended together and put through the masterGenerate function which generates the master string through the hash function. This masterString is then made into a master SecretKey which is used for encryption and exchanged to verify the correctness of either side's masterString.

```

encryptionKey = generateSecret(hmac("encryption", masterString));
macKey = hmac("macKey", masterString);

```

The encryptionKey and the macKey are created with the masterString after validation. After this step, the setup is finalized and the ATM and server can have secure and tamper free communications.

```

while (loggedIn == false) {
    encryptedIn = (byte[]) in.readObject();
    decodeDES(encryptedIn, encryptionKey);
    String[] ar = new String(decrypted).split("\\|");
    if (verifyMac(ar) == false) {
        System.out.println("Invalid message detected, closing connection");
        sock.close();
        return;
    }
    for (clientFile c : BankServer.getClients()) {
        String u = c.getLogin();
        if (u.compareTo(ar[0] + "|" + ar[1]) == 0) {
            message = "Login Successful" + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
            loggedIn = true;
            client = c;
            encodeDES(message, encryptionKey);
            out.writeObject(encrypted);
            break;
        }
    }
    if (loggedIn != true) {
        message = "Invalid Login" + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
        encodeDES(message, encryptionKey);
        out.writeObject(encrypted);
    }
}

```

This segment of code handles login of the client and verification of the login with the server. The client logs in on the ATM side and the server will receive the login and compare it

with all the clients that exist within the arrayList of clients. If the client does not log in successfully, the server will continue to listen for login attempts until one attempt is successful.

```
while (loggedIn == false) {
    System.out.println("Please enter your username:");
    String user = input.nextLine();
    System.out.println("Please enter your password:");
    String pass = input.nextLine();
    message = user + "|" + pass + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
    encodeDES(message, encryptionKey);
    out.writeObject(encrypted);
    encryptedIn = (byte[]) in.readObject();
    decodeDES(encryptedIn, encryptionKey);
    String[] ar = new String(decrypted).split("\\|");
    if (verifyMac(ar) == false) {
        System.out.println("Invalid message detected, closing connection");
        client.close();
        return;
    }
    if ("Invalid Login".compareTo(new String(ar[0])) != 0) {
        loggedIn = true;
        System.out.println("****" + ar[0] + "****");
        break;
    }
    System.out.println("****" + ar[0] + "****");
}
```

This segment of code is from the client side on the ATM and it handles the login portion of the project. It communicates with the server to validate the login and if unsuccessful, the loop will keep the program on the login screen.


```

while (true) {
    System.out.println("Please choose from the actions below by submitting a number (1, 2, 3, 4):");
    System.out.println("1: Deposit");
    System.out.println("2: Withdraw");
    System.out.println("3: Check Balance");
    System.out.println("4: Log out");
    int option = Integer.parseInt(input.nextLine());
    double amt;
    String m;
    switch (option) {
        case 1:
            System.out.println("How much do you want to deposit:");
            amt = Double.parseDouble(input.nextLine());
            m = "1" + "|" + Double.toString(amt) + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
            encodeDES(m, encryptionKey);
            out.writeObject(encrypted);
            break;
        case 2:
            System.out.println("How much do you want to withdraw:");
            amt = Double.parseDouble(input.nextLine());
            m = "2" + "|" + Double.toString(amt) + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
            encodeDES(m, encryptionKey);
            out.writeObject(encrypted);
            break;
        case 3:
            m = "3" + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
            encodeDES(m, encryptionKey);
            out.writeObject(encrypted);
            encryptedIn = (byte[]) in.readObject();
            decodeDES(encryptedIn, encryptionKey);
            String[] a = new String(decrypted).split("\\|");
            if (verifyMac(a) == false) {
                System.out.println("Invalid message detected, closing connection");
                client.close();
                return;
            }
            System.out.println("Current Balance: $" + a[0]);
            break;
        case 4:
            System.out.println("Thank you for choosing COE817 ATM");
            client.close();
            return;
    }
}

```

This big block of code is from the ATM client side and it handles the option selection. When deposit or withdrawal is chosen, another question prompts the client how much they wish to deposit or withdraw. After, the option number is appended with the amount along with a timestamp and the macKey for data validation. This is all then encoded using the encryptionKey derived from the masterString and then sent off to the server for validation.

```

while (true) {
    System.out.println("in");
    encryptedIn = (byte[]) in.readObject();
    decodeDES(encryptedIn, encryptionKey);
    String[] a = new String(decrypted).split("\\|");
    if (verifyMac(a) == false) {
        System.out.println("Invalid message detected, closing connection");
        sock.close();
        return;
    }
    String m;
    switch (Integer.parseInt(a[0])) {
        case 1:
            client.depositMoney(Double.parseDouble(a[1]));
            addToLog(client.getLogin().split("\\|")[0] + "|" + "Deposit");
            break;
        case 2:
            client.withdrawMoney(Double.parseDouble(a[1]));
            addToLog(client.getLogin().split("\\|")[0] + "|" + "Withdraw");
            break;
        case 3:
            m = Double.toString(client.getBalance()) + "|" + new Timestamp(System.currentTimeMillis()).getTime() + "|" + macKey;
            System.out.println(m);
            encodeDES(m, encryptionKey);
            out.writeObject(encrypted);
            addToLog(client.getLogin().split("\\|")[0] + "|" + "CheckBalance");
            break;
        case 4:
            sock.close();
            return;
    }
}

```

This final big chunk of code handles the user input for what options the user decides to take. Depending on the option, it will choose a case and perform the action. The server will also add the action to the log file and store it offline for security purposes. This logging function also encrypts the log data so that the log file is not in plain text format for security reasons.

Results

These are the results and screenshots of the GUI. It is a very basic banking application and the functionalities implemented were mentioned beforehand.

```
***** Welcome to the COE817 ATM *****
Please enter your username:
Andy
Please enter your password:
password
***Login Successful***
```

This is the main login screen. When logged in, it will return “login successful” and proceed to the main program.

```
***** Welcome to the COE817 ATM *****
Please enter your username:
Andy
Please enter your password:
pass
***Invalid Login***
Please enter your username:
```

This is the login screen when the login is unsuccessful. It will loop back to ask for login details again.

```
Please choose from the actions below by submitting a number (1, 2, 3, 4):
1: Deposit
2: Withdrawl
3: Check Balance
4: Log out
```

This is the main program screen. There are 4 options to choose from and to proceed to the next part, just enter a number corresponding with the options.

```
1
How much do you want to deposit:
10
Please choose from the actions below by submitting a number (1, 2, 3, 4):
1: Deposit
2: Withdrawl
3: Check Balance
4: Log out
3
Current Balance: $110.0
```

This is the deposit screen. It also shows the check balance to validate that the deposit worked.

```
2
How much do you want to withdraw:
10
Please choose from the actions below by submitting a number (1, 2, 3, 4):
1: Deposit
2: Withdrawl
3: Check Balance
4: Log out
3
Current Balance: $100.0
```

This is the withdrawal screen. It also shows the check balance to validate that the deposit worked.

```
4
Thank you for choosing COE817 ATM
```

This is the closing message for logging out of the ATM

Conclusion

This project helped enhance the knowledge on network security as well as working with different protocols and functions for key generation. Building on top of the given labs throughout the semester, this project encompassed all the skills learned within the semester with the added aspect of realism. Developing an authenticated key distribution protocol helped activate the critical thinking aspect on how a developer can further protect communication channels and the data that travels within it. Implementing nonces as well as timestamps were some of the key features of this project along with real time key generation and key derivation functions.